



Digital Fit

Aplicación web para gestión deportiva personalizada

Manuel Casinos Pérez

2º DAW · IES Conselleria

2026

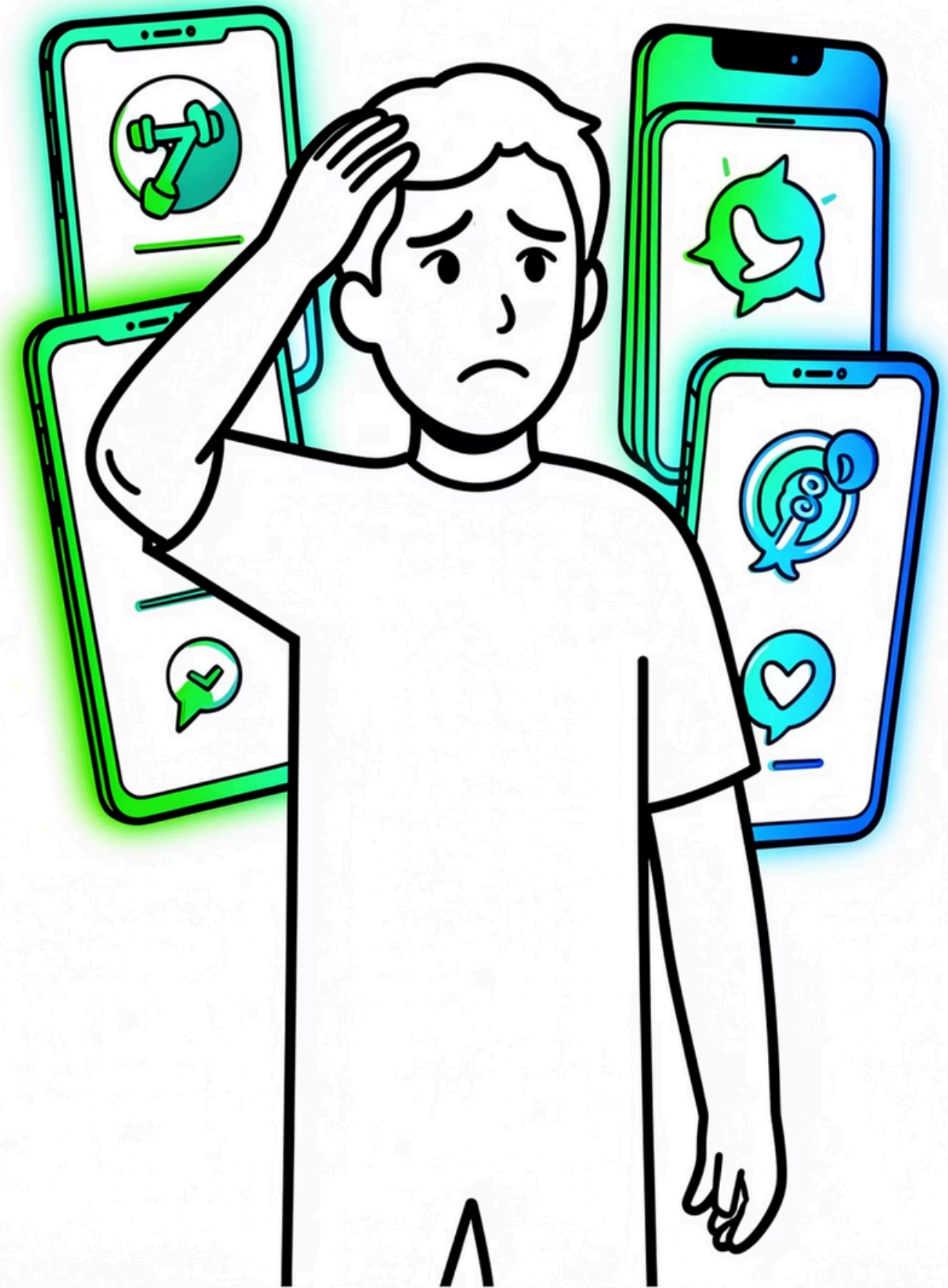
Origen de la idea y necesidad detectada

Motivaciones del Proyecto

- Unión de dos ámbitos que me interesan: deporte y programación
- Necesidad real observada en personas cercanas
- Interés por crear una solución útil, práctica y realista
- Oportunidad de desarrollar una app con valor técnico y funcional

Problema Detectado

- Muchas personas quieren entrenar pero no saben bien por dónde empezar
- La información está dispersa entre vídeos, redes sociales, mapas, notas y apps separadas
- No existe una herramienta cómoda que centralice entrenamientos, ubicaciones y seguimiento
- **No existe una sola app que reúna centros, lugares, entrenamientos, historial y estadísticas con un enfoque local para Valencia**



Digital Fit: Solución Propuesta

Aplicación web para gestión deportiva personalizada

Centraliza la actividad física en una sola plataforma, con enfoque local en [Valencia y la Comunidad Valenciana](#).



Autenticación y perfiles

Acceso seguro y datos personales.



Centros y lugares

Busca espacios deportivos cercanos.



Estadísticas y análisis

Datos claros sobre tu actividad.



Soporte y ayuda

Asistencia rápida cuando la necesites.



Entrenamientos personalizados

Rutinas adaptadas a cada usuario.



Historial y seguimiento

Registra tu progreso.



Valoraciones y comunidad

Comparte opiniones y experiencias.



Panel de administración

Gestión completa de usuarios y contenido.



Todo está integrado en un único sistema: cada usuario tiene su perfil, historial, entrenamientos, estadísticas y valoraciones, mientras que los administradores gestionan todo desde un panel centralizado conectado a la base de datos y a los roles de usuario.

Tecnologías elegidas y justificación

Criterio de selección del stack técnico

FRONTEND

- Angular
- TypeScript
- Leaflet (mapas)
- Bootstrap/CSS

Angular es lo que hemos aprendido en clase y me permite hacer una app que funciona sin recargar la página. TypeScript ayuda a que el código sea más ordenado y fácil de mantener.

BACKEND

- Spring Boot
- Spring Security
- JPA/Hibernate
- REST API

Spring Boot es el framework que usamos en el ciclo y me permite organizar el código de forma clara. Además, tiene todo lo que necesito para hacer una API REST sin complicarme.

BASE DE DATOS

- MySQL
- Relaciones normalizadas
- Usuarios como entidad central

MySQL es la base de datos que hemos usado siempre en clase. Como tengo muchas tablas relacionadas (usuarios, entrenamientos, centros...), MySQL es perfecto para esto.

DESPLIEGUE

- Docker Compose
- Nginx
- Contenedores

Docker me permite empaquetar todo (frontend, backend, base de datos) en contenedores para que funcione igual en cualquier ordenador. Nginx es el servidor web que sirve la aplicación.

El stack se eligió buscando equilibrio entre organización del proyecto, mantenibilidad, integración entre capas y adecuación a una aplicación web completa.

Arquitectura Cliente-Servidor

Cuando un usuario hace algo en la app, como crear un entrenamiento, por detrás pasa esto:



La seguridad y la lógica real están en el backend, no en el frontend.

Frontend: Angular → Services **Backend:** API REST → Controller → Service → Repository → **Base de datos:** MySQL



Si alguien intenta manipular Angular, el backend sigue comprobando que todo sea válido.

Seguridad, Calidad Técnica y Despliegue

Seguridad

- Spring Security con sesión por cookie
- BCrypt para encriptación de contraseñas
- Roles USER/ADMIN con control de acceso
- Guards en Angular para navegación
- Validación en backend, donde está la seguridad real



No uso JWT; Angular envía la cookie con withCredentials.

Calidad Técnica

- Arquitectura por capas: Controller → Service → Repository
- Separación clara entre lógica de negocio y acceso a datos
- Validación en múltiples niveles
- Manejo de excepciones

La aplicación está organizada para facilitar el mantenimiento, la depuración y la evolución del proyecto.

Despliegue Local

- Docker Compose para levantar todo junto
- Nginx como servidor web
- Contenedores para frontend y backend
- Base de datos MySQL en contenedor

El entorno local reproduce una ejecución real con frontend, backend y base de datos coordinados.

Si alguien manipula Angular, el backend sigue comprobando sesión, rol y propiedad.

Capturas

CONEXIÓN BACKEND

[illegible]

CONEXIÓN FRONTEND

```
C:\Users\Tamasdavid> cd Desktop\HMMU\PRAC7\CTS\Digital-Fit-Proyecto\Frontend\digital-fit
Initial chunk files | Name | Size |
main.js | main | 1.18 MB |
styles.css | styles | 29.87 KB |
| Initial total | 1.13 MB |

Application bundle generation complete. [31.401 seconds] - 2020-04-27T09:02:39.863Z

Watch mode enabled. Watching for file changes...
NOTE: The file size does not reflect development server per-request transformations.
- Press 'h' to help
- Press 'a' to show help
```

CONEXIÓN BD - MYSQL

```
src > main > resources > application.properties
1 spring.application.name=digital_fit
2 spring.datasource.url=jdbc:mysql://localhost:3306/digital_fit
3 spring.datasource.username=root
4 spring.datasource.password=root
5
6 spring.jpa.hibernate.ddl-auto=update
7 spring.jpa.defer-datasource-initialization=true
8 spring.sql.init.mode=always
9 spring.jpa.show-sql=true
10
11 #LOGS:
12
13 # Ruta de logs
14 logging.file.name=logs/digitalfit.log
15
16 # Nivel de logs
17 logging.level.root=INFO
18 logging.level.com.example.digital_fit=DEBUG
19
20 # Formato
21 logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss} - %msg%n
```

 DEMO EN VIVO

Demo en vivo

Recorrido funcional completo de Digital Fit

Ahora voy a enseñar el flujo real de la aplicación, no solo pantallas sueltas.



Login y autenticación



Navegación por rol: usuario vs admin



Crear y gestionar entrenamientos



Consultar historial de actividad



Visualizar estadísticas



Explorar mapa de centros y lugares



Acceder a soporte y valoraciones



Panel de administración si hay tiempo



CÓDIGO CLAVE

Código Clave y Estructura del Proyecto

Ubicación de componentes importantes

FRONTEND (Angular)

- app.routes.ts → Rutas de la aplicación
- auth-service.ts → Servicio de autenticación
- guards/ → Protección de rutas
- components/ → Componentes principales

BACKEND (Spring Boot)

- SecurityConfig.java → Configuración de seguridad
- CorsConfig.java → Configuración CORS
- controllers/ → Endpoints REST
- services/ → Lógica de negocio
- repositories/ → Acceso a datos

BASE DE DATOS

- Entidades JPA → Modelos de datos

Estas son las zonas principales que he detectado yo.



CONCLUSIÓN

Resultado Alcanzado y Visión de Futuro

Conclusión

- ✓ El proyecto cumple el objetivo: centralizar la gestión deportiva en una sola app
- ✓ Usuarios pueden entrenar de forma organizada y con seguimiento
- ✓ Administradores pueden gestionar contenido base, soporte y creación de administradores
- ✓ La arquitectura es escalable y segura
- ✓ El código está bien organizado y es mantenible

Mejoras Futuras

- 📱 Integración con dispositivos deportivos (smartwatch, pulseras de actividad)
- 🔔 Notificaciones push para recordar entrenamientos
- 🌐 Versión móvil nativa (iOS/Android)
- 🤝 Sistema de entrenador personal (conectar usuarios con entrenadores)
- 📊 Análisis más avanzados y recomendaciones personalizadas
- 🌍 Expansión a otras ciudades y comunidades autónomas